重要模块解释

controller.v

1. 模块端口定义（最外层）

```verilog
module controller(
    input [31:0] instr,        // 输入：当前要执行的 32 位指令

    // 输出：所有控制信号（指挥 CPU 各个部件）
    output reg reg_we,          // 寄存器写使能（1=允许写寄存器）
    output reg [1:0] reg_wd_sel,// 寄存器写数据选择（选 ALU 结果/存储器数据）
    output reg [1:0] reg_wa_sel,// 寄存器写地址选择（选 rt/rd）
    output reg alu_src_sel,     // ALU 第二个输入选择（寄存器/立即数）
    output reg [3:0] alu_op,    // ALU 操作码（加/减/或/lui）
    output reg ext_op,          // 立即数扩展方式（0=零扩展 1=符号扩展）
    output reg dm_we,           // 数据存储器写使能（1=写内存，sw 指令用）
    output reg j_en,            // j 跳转使能（1=执行无条件跳转）
    output reg beq_en           // beq 跳转使能（1=相等则跳转）
);
```

2. 提取指令关键字段

```verilog
wire [5:0] op;     // 指令高 6 位：操作码
wire [5:0] func;   // 指令低 6 位：功能码（R 型指令用）

assign op = instr[31:26];   // 取出指令[31:26] → 操作码
assign func = instr[5:0];   // 取出指令[5:0]  → 功能码
```

- MIPS 指令格式：**高 6 位 = op，区分指令类型**
- R 型指令（addu/subu）op=000000，靠 **func** 区分具体操作

3. 组合逻辑核心（always @(*)）

```verilog
always @(*) begin
    // 【默认值：所有信号先关闭，避免毛刺】
    reg_we = 1'b0;        // 不写寄存器
    reg_wd_sel = 2'b00;   // 默认写数据选 ALU
    reg_wa_sel = 2'b00;   // 默认写地址
    alu_src_sel = 1'b0;   // ALU B 端默认来自寄存器
    alu_op = 4'b0000;     // ALU 默认加法
    ext_op = 1'b1;        // 默认立即数符号扩展
    dm_we = 1'b0;         // 不写存储器
    j_en = 1'b0;          // 不跳转
    beq_en = 1'b0;        // 不执行 beq
```

作用：先把所有信号设为安全默认值，再根据指令覆盖修改，**防止电路产生错误电平**。

4. 根据指令操作码分发控制信号（case (op)）

（1）R 型指令：addu /subu

```verilog
6'b000000: begin    // R 型指令
    if(func == 6'b100001) begin // addu 无符号加法
        reg_we = 1'b1;     // 写寄存器
        reg_wa_sel = 2'b10;// 写地址 → rd
        alu_op = 4'b0000;  // ALU 做加法
    end
    else if(func == 6'b100011) begin // subu 无符号减法
        reg_we = 1'b1;
        reg_wa_sel = 2'b10;
        alu_op = 4'b0001;  // ALU 做减法
```

```
    end
end
```

（2）lui 指令

```verilog
6'b001111: begin    // lui 立即数加载到高位
    reg_we = 1'b1;        // 要写寄存器
    alu_src_sel = 1'b1;   // ALU B 端用立即数
    alu_op = 4'b0011;     // ALU 执行 lui 操作
    reg_wa_sel = 2'b01;   // 写地址 → rt
end
```

（3）ori 立即数或指令(ori $rt, $rs, immediate)

```verilog
6'b001101: begin    // ori
    reg_we = 1'b1;
    alu_src_sel = 1'b1;
    alu_op = 4'b0010;  // 或运算
    ext_op = 1'b0;     // 零扩展
    reg_wa_sel = 2'b01;
end
```

（4）lw 从内存读数据到寄存器(lw $t0/rt, 0($t1/rs))

```verilog
6'b100011: begin    // lw
    reg_we = 1'b1;
    alu_src_sel = 1'b1;
```

```
    alu_op = 4'b0000;     // 加法算地址
    reg_wd_sel = 2'b01;   // 写数据来自存储器
    reg_wa_sel = 2'b01;
end
```

（5）sw 把寄存器数据写入内存(sw $rt, 偏移($rs))

```verilog
6'b101011: begin    // sw
    dm_we = 1'b1;     // 使能内存写
    alu_src_sel = 1'b1;
    alu_op = 4'b0000;
end
```

（6）beq 条件跳转(beq $rs, $rt, 偏移量)

```verilog
6'b000100: begin    // beq
    beq_en = 1'b1;   // 开启 beq 跳转
    alu_op = 4'b0001;// ALU 做减法判断是否相等
end
```

（7）j 指令（你重点问的）(j 目标地址)

```verilog
6'b000010: begin    // j 无条件跳转
    j_en = 1'b1;     // 只需要开启跳转使能
end
```

alu.v

```verilog
module alu(
    input [31:0] A,      // 第一个数（来自 rs 寄存器）
    input [31:0] B,      // 第二个数（来自 rt 或 立即数）
    input [3:0] alu_op,  // 运算命令：算什么？加？减？或？lui？
    output reg [31:0] alu_out,  // 计算结果输出
    output zero          // 结果是不是 0？给 beq 用的！
);
```

解释端口：

- **A**：第一个操作数（rs 寄存器的值，32 位）
- **B**：第二个操作数（rt 或立即数，32 位）
- **alu_op**：**控制器发来的命令**，告诉 ALU 做什么运算
- **alu_out**：运算结果

zero：**结果等于 0 就输出 1，给 beq 判断是否相等**

```verilog
assign zero = (alu_out == 32'd0);
```

意思：

如果 **alu_out = 0** → zero = 1
否则 → zero = 0

用途：

专门给 beq 用！
A - B = 0 → 相等 → zero=1 → 跳转

```verilog
always @(*) begin
    case(alu_op)
```

意思：

根据 **alu_op（运算命令）** 选择做什么计算

```verilog
4'b0000: alu_out = A + B;      // addu 加法
```

命令 = 0000 → **A + B**
对应：**addu**

```verilog
4'b0001: alu_out = A - B;      // subu 减法
```

命令 = 0001 → **A - B**
对应：**subu、beq（判断相等）**

```verilog
4'b0010: alu_out = A | B;      // ori 按位或
```

命令 = 0010 → **A | B**
对应：**ori**

```verilog
4'b0011: alu_out = {B[15:0], 16'd0};// lui 高 16 位加载
```

命令 = 0011 → **把 B 的低 16 位放到高 16 位，低 16 位补 0**
对应：**lui**

```verilog
default: alu_out = 32'd0;
```

其他命令 → 输出 0

```verilog
endcase
end
endmodule
```

---

core_top.v

整个代码是什么？

这是一个**完整的单周期 CPU**
它能跑：
addu / subu / ori / lui / lw / sw / beq / j

一、先看所有线（wire）是什么

```verilog
// 数据通路
wire [31:0] pc_now;     // 当前指令地址（指令在哪一行）
wire [31:0] npc_out;    // 下一条指令地址
wire [31:0] instr;      // 指令本身
wire [4:0] rs, rt, rd;  // 寄存器地址
wire [31:0] gpr_rd1;    // 寄存器读出来的数据 1（rs）
wire [31:0] gpr_rd2;    // 寄存器读出来的数据 2（rt）
wire [31:0] ext_out;    // 扩展后的立即数
wire [31:0] alu_B;      // ALU 的第二个输入
wire [31:0] alu_result; // ALU 计算结果
```

```
wire zero;              // ALU 结果是否为 0（给 beq 用）
wire [31:0] dm_rd;       // 内存读出来的数据
wire [4:0] reg_wa;       // 寄存器写地址
wire [31:0] reg_wd;      // 寄存器写数据

// 控制信号（指挥各个部件）
wire reg_we;            // 寄存器写使能
wire [1:0] reg_wd_sel;  // 寄存器写数据选择
wire [1:0] reg_wa_sel;  // 寄存器写地址选择
wire alu_src_sel;       // ALU 输入选择：寄存器 or 立即数
wire [3:0] alu_op;       // ALU 运算命令
wire ext_op;            // 立即数扩展方式
wire dm_we;             // 内存写使能
wire j_en;              // 跳转使能
wire beq_en;            // 条件跳转使能
```

二、指令拆分（从指令里取出 rs、rt、rd）

```verilog
assign rs = instr[25:21];
assign rt = instr[20:16];
assign rd = instr[15:11];
```

从 32 位指令里抠出：

- **rs**：第 25~21 位
- **rt**：第 20~16 位
- **rd**：第 15~11 位

三、逐模块解释（最关键！）

1. PC 指令地址计数器

```verilog
pc u_pc(
    .clk(clk), .reset(reset),
    .npc_in(npc_out), .pc_out(pc_now)
);
```

作用：
存当前指令地址，每来一个时钟，更新为下一个地址。

---

2. NPC 下条指令地址计算

```verilog
npc u_npc(
    .pc_now(pc_now),
    .j_imm(instr[25:0]),
    .beq_imm(ext_out),
    .beq_en(beq_en & zero),
    .j_en(j_en),
    .npc_out(npc_out)
);
```

作用：

- 正常：`PC + 4`
- **beq**：相等就跳转
- **j**：无条件跳转

---

3. IM 指令存储器（存所有指令）

```verilog
im u_im(
    .pc(pc_now), .instr(instr)
);
```

作用：
根据地址，取出指令

---

4. CTRL 控制器（大脑！发命令）

```verilog
controller u_controller(
    .instr(instr),
    .reg_we(reg_we), .reg_wd_sel(reg_wd_sel), .reg_wa_sel(reg_wa_sel),
    .alu_src_sel(alu_src_sel), .alu_op(alu_op), .ext_op(ext_op),
    .dm_we(dm_we), .j_en(j_en), .beq_en(beq_en)
);
```

作用：
看指令 → 发控制信号
你之前问的所有信号都是这里发出来的！

---

5. GPR 寄存器堆（32 个寄存器）

```verilog
reg_file u_reg_file(
    .clk(clk), .reset(reset),
    .rs(rs), .rt(rt), .rd(reg_wa),
    .reg_we(reg_we), .wd(reg_wd),
    .rd1(gpr_rd1), .rd2(gpr_rd2)
);
```

作用：

- 读：`rs → rd1`，`rt → rd2`
- 写：`reg_we=1` 时，把 `reg_wd` 写入 `reg_wa`

---

6. EXT 立即数扩展

```verilog
ext u_ext(
    .imm_in(instr[15:0]), .ext_op(ext_op), .imm_out(ext_out)
);
```

作用：
16 位立即数 → 32 位

- `ext_op=0` → 零扩展（ori）
- `ext_op=1` → 符号扩展（lw/sw/beq）

---

7. ALU 计算器（你刚才看懂的那个！）

```verilog
alu u_alu(
    .A(gpr_rd1), .B(alu_B), .alu_op(alu_op),
    .alu_out(alu_result), .zero(zero)
);
```

作用：

- addu：加
- subu/beq：减
- ori：或
- lui：高 16 位加载

---

8. DM 数据存储器（内存）

```verilog
dm u_dm(
    .clk(clk), .reset(reset),
```

```
    .addr(alu_result), .wd(gpr_rd2), .dm_we(dm_we),
    .rd(dm_rd)
);
```

作用:

- **lw**: 读内存 → rd
- **sw**: 写 gpr_rd2 → 内存

npc.v

这是 **NPC 模块（下一条指令地址计算器）**
作用只有一个:
算出来 CPU 下一条该执行哪条指令!

```verilog
module npc(
    input [31:0] pc_now,    // 当前指令地址（现在在哪）
    input [25:0] j_imm,     // j 指令的 26 位跳转目标地址
    input [31:0] beq_imm,   // beq 跳转的偏移量
    input beq_en,          // beq 跳转使能（相等=1）
    input j_en,            // j 跳转使能
    output [31:0] npc_out   // 输出：下一条指令地址
);
```

---

1. 第一步：默认下一条 = PC + 4

```verilog
wire [31:0] pc_add4;
assign pc_add4 = pc_now + 4'd4;  // 正常执行：下一条地址
```

- 指令是 32 位（占 4 字节）
- 所以**下一条就是 +4**
- 没有跳转时，CPU 就走这个地址

---

2. 第二步：算 beq 跳转地址

```verilog
wire [31:0] beq_addr;
assign beq_addr = pc_add4 + (beq_imm << 2);
```

beq 跳转公式：

下一条地址 = PC+4 + 偏移量 × 4

为什么要 `<< 2`（左移 2 位 = ×4）？
因为指令地址**必须是 4 的倍数**。

---

3. 第三步：优先级判断（最核心！）

```verilog
assign npc_out =
    j_en       ? {pc_add4[31:28], j_imm, 2'b00} :  // 1. j 指令优先
    beq_en     ? beq_addr :                        // 2. 然后是 beq
                 pc_add4;                         // 3. 最后正常 PC+4
```

优先级：

j 无条件跳转 > beq 条件跳转 > 正常执行

ext.v

这是 **MIPS CPU 中非常核心的组合逻辑电路**，专门用来把 16 位的立即数扩展成 32 位，给 CPU 的运算、寻址使用，我给你逐行讲清楚，包括原理、代码含义、功能作用。

完整代码 + 中文注释版

```verilog
// 模块名：ext  功能：16 位立即数 → 32 位立即数（符号扩展/零扩展）
module ext(
    input  [15:0] imm_in,   // 输入：16 位原始立即数
    input         ext_op,   // 输入：扩展控制信号（0=零扩展，1=符号扩展）
    output [31:0] imm_out   // 输出：扩展后的 32 位立即数
);

// 三目运算符：根据 ext_op 选择扩展方式
assign imm_out = ext_op ? {{16{imm_in[15]}}, imm_in} :  // ext_op=1：符号扩展
                          {16'd0, imm_in};            // ext_op=0：零扩展

endmodule
```

逐行 / 逐段详细解释

1. 模块端口定义

```verilog
module ext(
    input  [15:0] imm_in,   // 16 位输入：CPU 指令中的立即数（比如加法、跳转的常数）
    input         ext_op,   // 1 位控制信号：决定用哪种扩展方式
    output [31:0] imm_out   // 32 位输出：给 CPU ALU、寄存器使用的扩展后数
);
```

- `imm_in`：**16 位立即数输入**，来自指令的低 16 位。
- `ext_op`：**扩展模式选择**，只有 0 和 1 两种取值。
- `imm_out`：**32 位输出**，CPU 内部运算必须用 32 位数据。

---

2. 核心逻辑：三目运算符

```verilog
assign imm_out = ext_op ? 符号扩展 : 零扩展;
```

这是 Verilog 的**组合逻辑赋值**，等价于：

> 如果 `ext_op = 1` → 执行**符号扩展**
> 如果 `ext_op = 0` → 执行**零扩展**

ext.v

1. 第一段：`mux2_32` —— 32 位 二选一选择器

```verilog
module mux2_32(     // 2 选 1 32 位多路选择器
    input [31:0] in0,   // 输入 0：32 位
    input [31:0] in1,   // 输入 1：32 位
    input sel,          // 选择信号：1 位
    output [31:0] out   // 输出：32 位
);
assign out = sel ? in1 : in0;
endmodule
```

功能

二选一开关

- 当 `sel = 0` → 输出 `in0`
- 当 `sel = 1` → 输出 `in1`

名字含义

- `mux2`：2 路输入
- `32`：**每一路都是 32 位宽**（比如 CPU 里的指令、数据、地址）

核心语句

```verilog
assign out = sel ? in1 : in0;
```

这是 Verilog 里的**三目运算符**，和 C 语言一样：
条件成立？选这个：否则选那个

---

2. 第二段：`mux3_5` —— 5 位 三选一选择器

```verilog
module mux3_5(      // 5 位 3 选 1 多路选择器
    input [4:0] in0,   // 输入 0：5 位
    input [4:0] in1,   // 输入 1：5 位
    input [4:0] in2,   // 输入 2：5 位
    input [1:0] sel,   // 选择信号：2 位（可以表示 00/01/10）
    output [4:0] out   // 输出：5 位
);
assign out = (sel==2'b00) ? in0 : (sel==2'b01) ? in1 : in2;
endmodule
```

功能

三选一开关

- `sel = 00` → 输出 `in0`
- `sel = 01` → 输出 `in1`
- `sel = 10` 或 `11` → 输出 `in2`

名字含义

- `mux3`：3 路输入
- `5`：**每一路都是 5 位宽**

5 位在 CPU 里专门用来表示**寄存器编号**（rt/rd 寄存器地址）

核心语句

```verilog
assign out = (sel==2'b00) ? in0 : (sel==2'b01) ? in1 : in2;
```

嵌套三目运算符，实现 **3 选 1**。

im.v

完整代码解释

```verilog
module im(
    input [31:0] pc,        // 输入：32 位程序计数器地址
    output [31:0] instr     // 输出：32 位 MIPS 指令
);
```

- 这是模块定义，模块名叫 `im`（指令存储器）。
- **输入**：`pc` 是 32 位的程序计数器，告诉 CPU **当前要执行哪条指令**。
- **输出**：`instr` 是读出的 **32 位 MIPS 机器指令**。

```verilog
wire [9:0] addr;
// PC 的低 10 位作为存储器地址
assign addr = pc[9:0];
```

- 因为存储器只有 **1024 字节**，只需要 **10 根地址线** 就能全部寻址（$2^{10} = 1024$）。
- 所以直接取 **PC 的低 10 位** 作为存储器的实际地址。

gpr.v

```verilog
// 模块名：gpr（General Purpose Register，通用寄存器组）
```

```
module gpr(
    input clk,           // 时钟信号（上升沿触发）
    input reset,         // 复位信号（高电平复位，所有寄存器清零）
    input [4:0] rs,      // 源寄存器 1 编号 指令[25:21]
    input [4:0] rt,      // 源寄存器 2 编号 指令[20:16]
    input [4:0] rd,      // 目的寄存器编号 指令[15:11]
    input reg_we,        // 写使能信号（1=允许写入，0=禁止写入）
    input [31:0] wd,     // 要写入的数据
    output [31:0] rd1,    // 源寄存器 1 读出的数据
    output [31:0] rd2     // 源寄存器 2 读出的数据
);

// 定义 32 个 32 位寄存器：regs[0] ~ regs[31]
reg [31:0] regs[31:0];
integer i;  // 循环变量，用于复位时清零所有寄存器

// 复位逻辑：复位信号上升沿 → 所有寄存器清零
always @(posedge reset) begin
    for(i=0;i<32;i=i+1) regs[i] <= 32'd0;
end

// 读寄存器：组合逻辑，随时读取，$0 恒等于 0
assign rd1 = (rs == 5'd0) ? 32'd0 : regs[rs];
assign rd2 = (rt == 5'd0) ? 32'd0 : regs[rt];

// 写寄存器：时钟上升沿触发，写入目的寄存器（$0 不可写入）
always @(posedge clk) begin
    if(reg_we && (rd != 5'd0))
        regs[rd] <= wd;
end
endmodule
```

dm.v

端口解释

```verilog
module dm(
    input clk,            // 时钟（单周期统一时钟）
    input reset,          // 高电平复位
    input [31:0] addr,     // 32 位访存地址（来自 ALU 计算结果）
    input [31:0] wd,       // 要写入 DM 的 32 位数据（来自寄存器 rt）
    input dm_we,          // 写使能：1=写（sw），0=读（lw）
    output [31:0] rd       // 从 DM 读出的 32 位数据（送给寄存器堆）
);
```

存储器定义

```verilog
reg [7:0] dm_mem[1023:0];
```

- **位宽**：每个单元 8 位（1 字节）
- **深度**：1024 个单元 → 总共 **1KB**
- 完全满足实验要求：**1KB 数据存储器**

复位逻辑

```verilog
always @(posedge reset) begin
    for(i=0;i<1024;i=i+1) dm_mem[i] <= 8'd0;
end
```

- 复位**上升沿**把所有存储单元清 0
- 保证上电 / 复位后数据存储器干净无脏值

写操作（sw 指令）

```verilog
```

```
always @(posedge clk) begin
    if(dm_we) begin
        dm_mem[addr[9:0]]   <= wd[31:24];
        dm_mem[addr[9:0]+1] <= wd[23:16];
        dm_mem[addr[9:0]+2] <= wd[15:8];
        dm_mem[addr[9:0]+3] <= wd[7:0];
    end
end
```

关键点：

- **时钟上升沿写入**（时序逻辑）
- **dm_we=1 才写** → 对应 sw 指令
- **addr[9:0]**：因为只有 1KB，只用低 10 位寻址
- **大端模式**：
  - 数据最高字节 → 存最低地址
  - 符合 MIPS 标准格式

六、读操作（lw 指令）

```verilog
assign rd = {
    dm_mem[addr[9:0]],
    dm_mem[addr[9:0]+1],
    dm_mem[addr[9:0]+2],
    dm_mem[addr[9:0]+3]
};
```

- **组合逻辑**：地址一到，数据立刻出来
- 把**连续 4 字节**拼成 **32 位字**
- 对应 lw 指令：从 DM 读数据到寄存器